

MongoDB Time-Series as a Backend for LuciadRIA WFS

Executive Summary

MongoDB time-series collections fit well behind a LuciadRIA WFS layer when the source data is append-heavy, time-stamped, and geospatial. In this pattern, MongoDB stores the authoritative vessel history, while a WFS service exposes the features LuciadRIA needs for map display, filtering, styling, and temporal navigation.

Recommended Data Flow

1. Ingest ship position updates into a MongoDB time-series collection.
2. Group records by vessel using `suid` as the meta field.
3. Store event time in `reported_time_info` as the time field.
4. Index historical access by ship, time, and geometry.
5. Publish a WFS endpoint that reads from MongoDB and returns feature data to LuciadRIA.

In this repository, the time-series collection is `tracks_local_timeseries` inside the `ship_tracking` database.

Why Time-Series Works Here

MongoDB time-series collections are a strong fit for vessel tracking because they optimize for:

- high-frequency writes,
- per-vessel historical ordering,
- time-range queries,
- spatial lookups on reported positions,
- and compact storage of repeated measurements.

For a ship-tracking backend, that means the database can preserve a full movement history without forcing the application to manage a separate ledger table or external archive.

LuciadRIA WFS Consumption Pattern

LuciadRIA can consume WFS features as a client-side GIS layer. The WFS service should expose the ship records in a feature format that LuciadRIA can style, filter, and query.

The practical architecture is:

- MongoDB time-series collection for historical ship events.
- A feature-service or WFS adapter that translates MongoDB documents into WFS feature responses.
- LuciadRIA as the visualization and interaction layer.

This keeps MongoDB focused on storage and retrieval while LuciadRIA handles rendering, selection, and user interaction.

Repository-Aligned Implementation Notes

The current proof of concept already supports the backend pattern needed for this architecture:

- `reported_time_info` is used as the primary time-sorting field.
- `tracks_local_timeseries` is created as a MongoDB time-series collection.
- Secondary indexes support ship history by `suid`, `mmsi_number`, and `trackLocation`.
- The API already queries the time-series collection for polygon-based overlays.

This means the repository can back both current-state ship views and historical track views from the same persisted time-series data.

Suggested WFS Design

For LuciadRIA, the WFS layer should expose two useful views:

- latest vessel position per ship for live mapping,
- historical track segments for time-aware replay or analysis.

That approach keeps the client simple while preserving the full value of the underlying time-series store.

Key Benefit

The main advantage of this design is separation of concerns:

- MongoDB time-series handles ingestion and historical storage.
- WFS provides an interoperable geospatial feature interface.
- LuciadRIA provides the interactive GIS client.

This is a clean fit when the goal is to serve maritime tracks to a geospatial front end without flattening the data model or losing history.

Notes

This brief is a technical overview based on the current ship-tracking proof of concept and should be treated as implementation guidance rather than a formal product specification.